Embedded Linux Engineer Assignment

Embedded Linux for Virtual Device

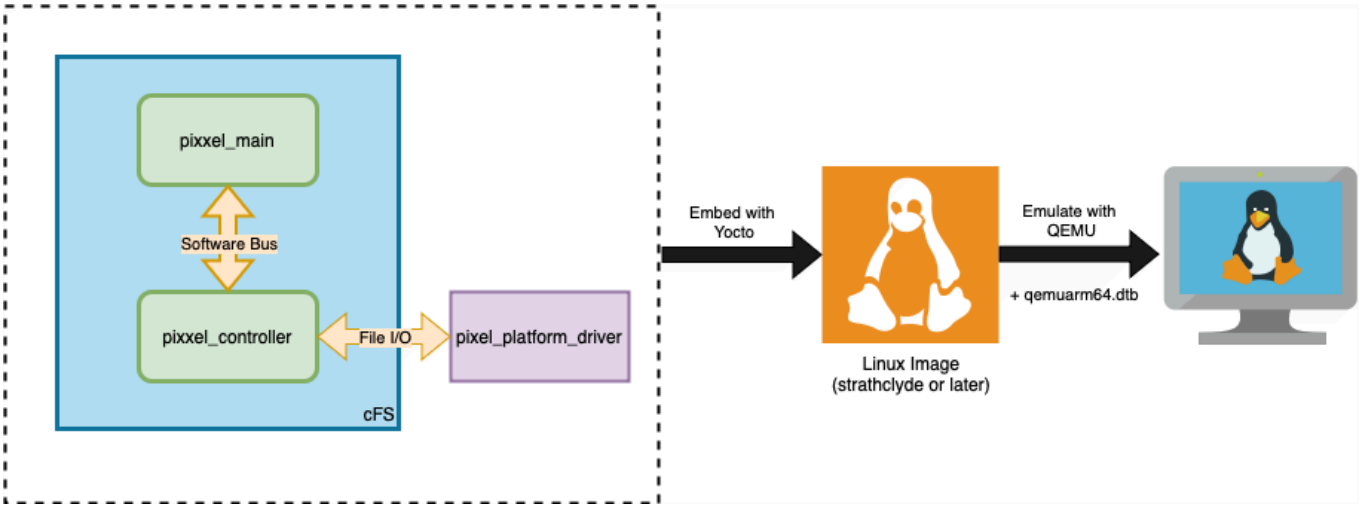
Introduction

Building Linux images with custom firmware and software is an integral part of the workflow of an embedded engineer. This assignment covers a broad range of topics required for such a workflow including the Yocto build system, device trees, platform device drivers, user space applications, and emulation.

---

1. Assignment

In this assignment, you are to develop a custom Linux image containing a driver for a virtual platform device and a user space application that interacts with the driver. Each component of the deliverables are outlined below and also represented in this diagram.



1.1 Virtual Platform Device and Device Tree

A "platform" device is a device that is directly integrated into the hardware platform, whereas a virtual platform device is a digital replica of a platform device that behaves in a similar manner but is not backed by any real hardware.

In this assignment, we will be working with ARM64 machines which typically use a device tree to describe platform devices. The device tree containing the virtual platform device is already given to you (`qemuarm64.dtb`) and the device node itself is described here:

```
pixxel-virt-dev@60000000 {
    compatible = "pixxel,virt-dev";
```

```
    reg = <0x00 0x60000000 0x00 0x00001000>;
};
```

Further, you can assume that the device contains the following registers. These registers are all virtual and need to be implemented in the driver design. The device tree node for the device only provides a register space to accommodate these registers.

Register	Address	Description
Enable(W)	0x60000000	Enable register for the device. Write `1` to enable the device and `0` to disable it.
Status(R)	0x60000004	Status register for the device. Reflects the status of the write-only Enable register ~50 ms after the write happens to the Enable register.

1.2 Platform Driver

A driver is a specialized software program that acts as a bridge between a computer's operating system and a connected hardware device. You are expected to write a driver for the virtual platform device described. The driver should search the device tree for the device and expose an interface which will later be used by the application.

1.3 User-Space Application

For this section of the assignment, we would like to evaluate your ability to work with software frameworks. The framework of choice is NASA's Core Flight System (cFS). cFS is a platform-independent, reusable software framework designed to expedite flight software development. cFS achieves this by providing a set of five services that are the functional building blocks to create and host applications: Executive Service (ES), Software Bus (SB) Service, Event Service (EVS), Table Service (TBL), and Time Service (TIME). Here, we will only be focusing on the SB service. Documentation for the SB service can be found in sections 1.14 and 3.6 of the attached *Core Flight Executive Users Guide*.

Using the `sample_app` as a reference, design two simple apps: `pixxel_main` and `pixxel_controller` with the following specifications:

- `pixxel_controller` serves as the controller for the driver designed earlier and also acts on and acknowledges the commands provided to it by `pixxel_main`.
- `pixxel_main` has to:
  - start execution *after* `pixxel_controller` has started.
  - upon execution, command `pixxel_controller` to enable the device using the driver.
  - receive acknowledgment from `pixxel_main`.

    - wait for fifty milliseconds and then command `pixxel_controller` again to retrieve the status of the device.
    - check the status of the device and ensure that it is enabled.
- `pixxel_main` and `pixxel_controller` always interact using the Software Bus service.

1.4 Yocto (`poky`) and Linux Image

Yocto is an open-source Linux distribution and build system that is commonly used to generate Linux images. For this assignment, you can assume that the baseline is Yocto's (`poky`) latest branch (*`strathclyde`* at the time of designing this assignment; use whichever branch is the latest today). Starting from this baseline, create a custom Board Support Package (BSP) layer named `meta-pixxel` that contains two recipes: One to embed the platform driver into the Linux image and the other to embed the user-space application into the same image. Finally, since the device is being emulated using Quick EMUlator (QEMU), ensure the build targets the `qemuarm64` device.

1.5 Emulation

QEMU is an emulator that allows you to run a complete Linux image on your build system. The Yocto Project provides its own version (a wrapper) of QEMU that is invoked by using the `runqemu` command after a build. In this assignment you will need to use `runqemu` along with the provided `qemuarm64.dtb` device tree on your build to test the functionality of your driver and user-space application. The appropriate method to achieve this can be found in `runqemu` 's help.

---

2. Requirements

The requirements of the assignment are summarized in this section.

1. Design a platform device driver for registering the device and simulating the functionality of the registers described in the previous section. The device tree binary is already provided to you.
2. Create two user-space applications using NASA's cFS framework for testing the functionality of the device driver.
3. Using Yocto's latest branch as the baseline, create a BSP layer that provides recipes to embed the device driver and the user-space applications into a Linux image targeting the `qemuarm64` `MACHINE` .
4. Use the `runqemu` tool along with the appropriate settings to emulate the `qemuarm64` machine using the `qemuarm64.dtb` device tree binary and clearly print results from the two

applications in cFS.

---

3. Deliverables

The deliverables are:

- Source code for the device driver.
- Source code for the user-space applications that are well-organized within cFS.
- The Yocto BSP layer with well-organized recipes.
- Emulation output
- Short write-up (not more than a page) on challenges and implementation

---

4. Evaluation Criteria

1. **Completion** – The assignment covers a broad range of topics. However, the candidate is not expected to have immediate knowledge of all the relevant topics—it is acceptable if only some parts of the assignment are completed. The evaluation will be conducted on the basis of the candidate's ability to plan and execute. For instance, successfully compiling the device driver and / or user-space applications natively might be a better first step instead of diving head first into Yocto BSP layer development which may yield no results.
2. **Correctness** – Does the design produce expected results?
3. **Quality** – Is the code well-structured (including Yocto BSP and cFS applications) and readable?
4. **Simulation & Emulation** – Was the platform device simulated correctly in the driver and does the emulation show the expected results?
5. **Differentiation** – Based on your own experience in this field, include any relevant improvements to the design which are worth consideration.

Good luck, and happy coding! 🚀